

Project title: Mini Internet Protocol Stack Simulator (Python)

Deadline: 24/5/2026, 23:59

Goal:

Implementing a simplified network simulator that demonstrates how Layer 2, Layer 3, and Layer 4 work together to deliver data from one host to another.

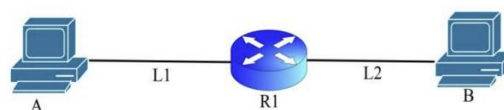
Group size: Up to 2 students

Project description:

In this project, students must implement a simplified network stack consisting of Data Link (Layer 2), Network (Layer 3), and Transport (Layer 4) headers. Each layer has its own set of fields, and data must be encapsulated as it moves down the stack.

Network Topology:

The network consists of two hosts (A and B) connected via a router (R1):



IP Addressing Scheme:

The network is divided into two subnets:

- Network 1: 10.0.1.0/24
- Network 2: 10.0.2.0/24

The IP addresses of the devices are defined as follows:

- Host A: 10.0.1.10
- Router R1 (Interface 1): 10.0.1.1
- Router R1 (Interface 2): 10.0.2.1
- Host B: 10.0.2.20

MAC Addressing Scheme:

Each network interface is assigned a unique MAC address:

- Host A: AA:AA:AA:AA:AA:AA
- Router R1 (Interface 1): BB:BB:BB:BB:BB:BB
- Router R1 (Interface 2): CC:CC:CC:CC:CC:CC
- Host B: DD:DD:DD:DD:DD:DD

Protocol Header Definitions:

- Layer 2: Data Link Layer (Ethernet-like Frame)
 - ❖ Destination MAC (6 bytes) → e.g., BB:BB:BB:BB:BB:BB
 - ❖ Source MAC (6 bytes) → e.g., AA:AA:AA:AA:AA:AA
 - ❖ Type (2 bytes) → 0x0800 (indicates IPv4 payload)
 - ❖ Payload (variable) → contains the Layer 3 (IP) packet
- Layer 3: Header (IP-like Packet)
 - ❖ Source IP (4 bytes) → e.g., 10.0.1.10
 - ❖ Destination IP (4 bytes) → e.g., 10.0.2.20
 - ❖ TTL (1 byte) → 100 (decremented at each router)
 - ❖ Protocol (1 byte) → 17 (indicates UDP payload)
 - ❖ Total Length (2 bytes) → e.g., 40 (header + payload size)
 - ❖ Payload (variable) → contains the Layer 4 (UDP segment)
- Layer 4: Header (UDP-like Segment with ACK support)
 - ❖ Source Port (2 bytes) → e.g., 5000
 - ❖ Destination Port (2 bytes) → e.g., 80
 - ❖ Length (2 bytes) → e.g., 100 (header + data)
 - ❖ Checksum (2 bytes) → e.g., computed value for error detection
 - ❖ Type (1 byte) → 0 = DATA, 1 = ACK
 - ❖ Sequence Number (1 byte) → e.g., 0 or 1
 - ❖ Data (variable) → contains the application message (empty for ACK)

Functionalities:

- Layer 2: Data Link

The Data Link Layer is responsible for frame-based communication over a local network. It performs framing, MAC addressing and local delivery.

Required functionalities:

- ❖ Framing & Encapsulation: Encapsulate the Layer 3 packet into a frame before transmission.
 - ❖ MAC Addressing: Use source and destination MAC addresses to deliver frames within the local network. Each node must maintain a table that maps the next hop IP address (provided by the network layer) to the MAC addresses. This table is used to determine the destination MAC address when sending a frame.
 - ❖ Switching / Forwarding:
 - o Learn the incoming source MAC address from received frames
 - o Forward frames to the destination MAC address
 - ❖ Frame Delivery: Deliver valid payload (IP packet) to Layer 3 at the receiver.
- Layer 3: Network

The Network Layer is responsible for packet-based communication across networks. It performs logical addressing, routing, and forwarding of packets between different networks.

Required functionalities:

- ❖ Encapsulation: Encapsulate the Layer 4 segment into an IP-like packet before transmission.
- ❖ IP Addressing: Use source and destination IP addresses to identify hosts across networks.
- ❖ Routing / Forwarding: Routers must forward packets based on a routing table. Each node must maintain a routing table that maps the destination IP address to the outgoing interface and the next-hop IP addresses. This table is used to determine how packets are forwarded. As an example, when R1 receives the packet on Interface 1 from host A, it reads the destination IP address (10.0.2.20) and performs a routing table lookup. It finds that the destination network (10.0.2.0/24) is directly connected via Interface 2 (10.0.2.1). Since the destination is on a directly connected network, the next-hop IP is set to the destination IP itself (10.0.2.20). The router then selects Interface 2 as the outgoing interface and forwards the packet to the data link layer **along with the next hop IP address and the selected interface for transmission**.
- ❖ TTL Handling:
 - Decrement TTL at each router
 - Drop the packet if TTL reaches 0
- ❖ Packet Delivery: Deliver valid payload (UDP-like segment) to Layer 4 at the destination host.

➤ Layer 4: Transport (UDP-like Segment with ACK – rdt2.2)

The Transport Layer is responsible for end-to-end communication between applications. It performs port-based delivery, segmentation, error detection, and reliable data transfer using a simple acknowledgement mechanism.

Required functionalities:

- ❖ Segmentation & Encapsulation: Encapsulate application data into a UDP-like segment before transmission.
- ❖ Port-based Communication: Use source and destination port numbers to identify sending and receiving applications.
- ❖ Length Handling: Compute and include the total segment length (header + data).
- ❖ Checksum / Error Detection:
 - Compute a checksum for each segment
 - Verify the checksum at the receiver
 - Discard corrupted segments
- ❖ Reliable Data Transfer (rdt2.2 – Alternating Bit Protocol):
 - Use a sequence number (0 or 1) for each DATA segment

- The sender alternates sequence numbers: $0 \rightarrow 1 \rightarrow 0 \rightarrow 1$
- Upon receiving a valid DATA segment:
 - The receiver delivers the data to the application
 - The receiver sends an ACK with the same sequence number
- If a segment is corrupted or a duplicate:
 - The receiver re-sends the last ACK
- The sender:
 - Waits for the correct ACK
 - If an incorrect or duplicate ACK is received, the sender retransmits the current segment

❖ Data Delivery: Deliver valid data to the application layer at the receiver.

Output / Logging Requirement:

The implementation must produce clear output (e.g., printed logs) showing the operation of each layer.

➤ Layer 2: Data Link

- ❖ When a frame is received
- ❖ When a frame is created and sent
- ❖ When a MAC address is learned
- ❖ When a frame is forwarded
- ❖ When a packet is delivered to the Network Layer
- ❖ When a packet is received from the Network Layer

➤ Layer 3: Network

- ❖ When a packet is forwarded to the Data Link Layer
- ❖ When a packet is received from the Data Link Layer
- ❖ When a segment is received from the Transport Layer
- ❖ When the routing decision is made
- ❖ When TTL is decremented or a packet is dropped due to TTL expiry
- ❖ When a segment is delivered to the Transport Layer

➤ Layer 4: Transport

- ❖ When a segment is received from the Network Layer
- ❖ When a segment is sent to the Network Layer

- ❖ When data is received from application layer
- ❖ When the checksum is computed and verified
- ❖ When a segment is discarded due to checksum error
- ❖ When data is delivered to the application layer
- ❖ When an ACK is sent (including sequence number)
- ❖ When an ACK is received
- ❖ When a segment is retransmitted due to incorrect ACK

The following is the expected output log that must be produced for sending data with the size of 10 bytes from Host A to Host B

Host A: Layer 4: Data received from Application Layer. Data size=10

Host A: Layer 4: Checksum computed

Host A: Layer 4: Segment created by adding transport layer header (DATA, seq=0) (encapsulation)

Host A: Layer 4: Segment sent to Network Layer

Host A: Layer 3: Segment received from Transport Layer: SRC_IP=10.0.1.10, DST_IP=10.0.2.20, TTL=100

Host A: Layer 3: Destination IP read: 10.0.2.20

Host A: Layer 3: Routing table lookup performed

Host A: Layer 3: Next-hop IP determined: 10.0.1.1

Host A: Layer 3: Outgoing interface selected

Host A: Layer 3: Packet forwarded to Data Link Layer

Host A: Layer 2: Packet received from Network Layer

Host A: Layer 2: Destination MAC lookup for next-hop IP (10.0.1.1) → BB:BB:BB:BB:BB:BB

Host A: Layer 2: Frame created: SRC_MAC=AA:AA:AA:AA:AA:AA, DST_MAC=BB:BB:BB:BB:BB:BB

Host A: Layer 2: Frame sent

Router R1: Layer 2: Frame received on Interface 1

Router R1: Layer 2: Source MAC learned: AA:AA:AA:AA:AA:AA on Interface 1

Router R1: Layer 2: Packet delivered to Network Layer

Router R1: Layer 3: Packet received from Data Link Layer: SRC_IP=10.0.1.10, DST_IP=10.0.2.20, TTL=100

Router R1: Layer 3: Destination IP read: 10.0.2.20

Router R1: Layer 3: TTL decremented: 100 → 99

Router R1: Layer 3: Routing table lookup performed

Router R1: Layer 3: Next-hop IP determined: 10.0.2.20

Router R1: Layer 3: Outgoing interface selected (Interface 2)

Router R1: Layer 3: Packet forwarded to Data Link Layer

Router R1: Layer 2: Packet received from Network Layer

Router R1: Layer 2: Destination MAC lookup for next-hop IP (10.0.2.20) → DD:DD:DD:DD:DD:DD

Router R1: Layer 2: Frame created: SRC_MAC=CC:CC:CC:CC:CC:CC, DST_MAC=DD:DD:DD:DD:DD:DD

Router R1: Layer 2: Frame forwarded on Interface 2

Host B: Layer 2: Frame received

Host B: Layer 2: Source MAC learned: CC:CC:CC:CC:CC:CC

Host B: Layer 2: Packet delivered to Network Layer

Host B: Layer 3: Packet received from Data Link Layer: SRC_IP=10.0.1.10, DST_IP=10.0.2.20, TTL=99

Host B: Layer 3: Destination IP read: 10.0.2.20

Host B: Layer 3: Packet identified as local delivery

Host B: Layer 3: Segment delivered to Transport Layer

Host B: Layer 4: Segment received from Network Layer

Host B: Layer 4: Checksum verified

Host B: Layer 4: DATA segment delivered to Application Layer. Data size=10

Host B: Layer 4: Segment created by adding transport layer header (ACK, seq=0)

Host B: Layer 4: Segment sent to Network Layer

Host B: Layer 3: Segment received from Transport Layer: SRC_IP=10.0.2.20, DST_IP=10.0.1.10, TTL=100

Host B: Layer 3: Destination IP read: 10.0.1.10

Host B: Layer 3: Routing table lookup performed

Host B: Layer 3: Next-hop IP determined: 10.0.2.1

Host B: Layer 3: Outgoing interface selected

Host B: Layer 3: Packet forwarded to Data Link Layer

Host B: Layer 2: Packet received from Network Layer

Host B: Layer 2: Destination MAC lookup for next-hop IP (10.0.2.1) → CC:CC:CC:CC:CC:CC

Host B: Layer 2: Frame created: SRC_MAC=DD:DD:DD:DD:DD:DD, DST_MAC=CC:CC:CC:CC:CC:CC

Host B: Layer 2: Frame sent

Router R1: Layer 2: Frame received on Interface 2

Router R1: Layer 2: Source MAC learned: DD:DD:DD:DD:DD:DD on Interface 2

Router R1: Layer 2: Packet delivered to Network Layer

Router R1: Layer 3: Packet received from Data Link Layer: SRC_IP=10.0.2.20, DST_IP=10.0.1.10, TTL=100

Router R1: Layer 3: Destination IP read: 10.0.1.10

Router R1: Layer 3: TTL decremented: 100 → 99

Router R1: Layer 3: Routing table lookup performed

Router R1: Layer 3: Next-hop IP determined: 10.0.1.10

Router R1: Layer 3: Outgoing interface selected (Interface 1)

Router R1: Layer 3: Packet forwarded to Data Link Layer

Router R1: Layer 2: Packet received from Network Layer

Router R1: Layer 2: Destination MAC lookup for next-hop IP (10.0.1.10) → AA:AA:AA:AA:AA:AA

Router R1: Layer 2: Frame created: SRC_MAC=BB:BB:BB:BB:BB:BB, DST_MAC=AA:AA:AA:AA:AA:AA

Router R1: Layer 2: Frame forwarded on Interface 1

Host A: Layer 2: Frame received

Host A: Layer 2: Source MAC learned: BB:BB:BB:BB:BB:BB

Host A: Layer 2: Packet delivered to Network Layer

Host A: Layer 3: Packet received from Data Link Layer: SRC_IP=10.0.2.20, DST_IP=10.0.1.10, TTL=99

Host A: Layer 3: Destination IP read: 10.0.1.10

Host A: Layer 3: Packet identified as local delivery

Host A: Layer 3: Segment delivered to Transport Layer

Host A: Layer 4: Segment received from Network Layer

Host A: Layer 4: Checksum verified

Host A: Layer 4: ACK received: seq=0

Execution Requirements:

The system will be evaluated in a controlled environment consisting of Host A, Router R1, and Host B as defined in the topology. The implementation must accept the application message size (in bytes) as a command-line argument (e.g., `python main.py 100` for a 100-byte message). It is assumed that there is no packet loss, no frame corruption, and all transmissions are deterministic. The maximum **data size** for a UDP-like segment is limited to 500 bytes (maximum application data per segment). Therefore, if the input message size exceeds this limit, the message must be segmented into multiple transport-layer segments. Each segment

must be transmitted sequentially using the rdt2.2 (week 8 lecture) protocol (sequence numbers 0 and 1), with the receiver sending an ACK for each correctly received segment before the sender proceeds. The name of the main file used for testing must be **main.py**. Students must implement a logical simulation only. The use of external libraries is not permitted. Only Python standard libraries (e.g., sys) may be used. The use of networking libraries such as socket is not allowed.

Code Structure:

The submission must follow a simple project structure consisting of 1) a main entry file **main.py** (which is the only file called for execution), 2) a **protocol.py** file containing the header definitions and classes for Layers 2, 3, and 4, 3) a **devices.py** file implementing the Host and Router classes, 4) a **config.py** file defining fixed parameters such as IP addresses, MAC addresses, routing tables, and 5) a README.md file providing a brief explanation of the implementation and instructions for running the program. The code must be well-documented, with clear comments provided for all classes and functions to explain their purpose and functionality.

Marking Rubric (Total: 100 Marks):

- Layer 2 – Data Link Layer (20 Marks): Marks are awarded for the correct implementation of Data Link Layer functionalities, including frame creation, MAC addressing, forwarding behaviour, and delivery of data to the upper layer.
- Layer 3 – Network Layer (20 Marks): Marks are awarded for the correct implementation of Network Layer functionalities, including packet creation, routing decisions, forwarding between networks, and basic control mechanisms such as TTL handling.
- Layer 4 – Transport Layer (25 Marks): Marks are awarded for the correct implementation of Transport Layer functionalities, including data segmentation, construction of transport segments, error detection, and the use of a simple reliable data transfer mechanism.
- End-to-End Functionality (15 Marks): Marks are awarded for the overall correctness of the system in delivering data from the sender to the receiver, including handling acknowledgements and correctly processing messages of different sizes.
- Output / Logging (10 Marks): Marks are awarded for producing clear and structured output that reflects the behaviour of the system at different layers. The logs should demonstrate the flow of data through the network and the key operations performed at each stage.
- Code Structure and Documentation (10 Marks): Marks are awarded for maintaining a clear and organized code structure, as well as providing appropriate documentation.

Submission Requirements:

Each group must submit two files: (1) a compressed file (ZIP) containing the complete source code, and (2) a document explaining the structure and design of the code. The document must clearly describe the organization of the files and the purpose of the main components of the implementation. The document must also include the full names of both group members. **Only one member of the group should submit the files on behalf of the group; duplicate submissions from both members are not permitted.** The LMS submission portal will open on 10 May. The submission file must be named in the format SID1-SID2.pdf and SID1-SID2.zip, where SID represents the student ID.